

Backend Documentation

Overview / Introduction

This is a backend for a Lost & Found application. Built with Node.js, Express, Prisma, PostgreSQL, and JWT authentication.

Setup / Installation

1. Clone repo: <https://github.com/TRakhym/lost-and-found-project-backend.git>
2. **cmd:** npm install
3. Create .env file with Environment Variables
4. **cmd:** npx prisma generate
5. Local PostgreSQL Database:
 - **cmd:** npx prisma migrate devRemote PostgreSQL Database:
 - **cmd:** npx prisma migrate deploy
6. **cmd:** npm run dev

Environment Variables

- With Remote PostgreSQL Database:
`DATABASE_URL="postgresql://postgres:72740205@localhost:5432/project-backend-database?schema=public"`
`JWT_SECRET=MY_CODE`
`PORT=3000`
- With Local PostgreSQL Database:
`DATABASE_URL="postgresql://project_backend_database_user:zIoxwxgY67XsGXyXUo3ANRcEOgU6R3tK@dpg-d6e2ifkr85hc73c7qrgg-a.oregon-postgres.render.com/project_backend_database"`
`JWT_SECRET=MY_CODE`
`PORT=3000`

API Endpoints / Routes

Auth

1. **POST** - api/auth/register

Description: Register a new user

Body -json:

```
{
  "email": "john@example.com",
  "password": "Password24",
  "name": "John"
}
```

Response:

```
{
  "token": "jwt_token_here",
  "user": {
    "email": "john@example.com",
    "name": "John doe"
  }
}
```

2. **POST** - api/auth/login

Description: Login user and get JWT token

Body -json:

```
{
  "email": "john@example.com",
  "password": "123456"
}
```

Response:

```
{
  "token": "jwt_token_here",
  "user": {
    "email": "john@example.com",
    "name": "John doe"
  }
}
```

Users

3. GET - api/users/profile

Description: Get information about user

Response:

```
{  
  "email": "john@example.com",  
  "name": "John Doe"  
  "phone": "87007007070",  
  "avatarUrl": "img_url_cloudinary"  
}
```

4. PUT - api/users/update

Description: Update user information

Body -form-data:

-name: "John Doe",

-phone: "87007007070"

-profilePic: files

Response:

```
{  
  "email": "john@example.com",  
  "name": "John Doe"  
  "phone": "87007007070",  
  "avatarUrl": "img_url_cloudinary"  
}
```

Items

5. **POST** - api/items

Description: Create new Item

Body -form-data:

-title: "Lost black wallet",
-description: "Small black leather wallet"
-type: "LOST"
-location: "Almaty",
-date: "2026-03-15"
-category: "accessories"
-images: files, – max images can be 5

Response:

```
{  
    "itemId": "1"  
}
```

6. **GET** - api/items

Description: Get all items

Response:

```
[  
    {  
        "itemId": "1",  
        "title": "Lost black wallet",  
        "description": "Small black leather wallet",  
        "type": "LOST",  
        "location": "Almaty",  
        "date": "2026-03-15",  
        "category": "accessories",  
        "images": ["img_url_1", "img_url_2"]  
    }  
]
```

7. **GET** - api/items/{id}

Description: Get single item by id

Response:

```
{  
    "itemId": "1",  
    "title": "Lost black wallet",  
    "description": "Small black leather wallet",  
    "type": "LOST",  
    "location": "Almaty",  
    "date": "2026-03-15",  
    "category": "accessories",  
    "images": ["img_url_1", "img_url_2"],  
    "user": {  
        "name": "John Doe",  
        "email": "john@example.com"  
    }  
}
```

```
}
```

8. **GET** - api/items/my

Description: Get items created by current user

Response:

```
[  
  {  
    "itemId": "1",  
    "title": "Lost black wallet",  
    "type": "LOST",  
    "location": "Almaty",  
    "date": "2026-03-15"  
  }  
]
```

9. **DELETE** - api/items/{id}

Description: Delete item

Response:

```
{  
  "message": "deleted"  
}
```

Claims

10. **POST** - api/items/{itemId}/claim

Description: Create claim for item

Body -json:

```
{
  "message": "any_message"
}
```

Response:

```
{
  "claimId": "2",
  "status": "PENDING"
}
```

11. **GET** - api/items/{itemId}/claims

Description: Get all claims for specific item (owner only)

Response:

```
[
  {
    "claimId": "2",
    "message": "This is my item",
    "status": "PENDING",
    "user": {
      "name": "John Doe",
      "email": "john@example.com"
    }
  }
]
```

12. **GET** - api/claims/my

Description: Get claims created by current user

Response:

```
[
  {
    "claimId": "2",
    "itemId": "1",
    "status": "PENDING",
    "message": "This is my item"
  }
]
```

13. **PATCH** - api/claims/{id}

Description: Update claim status (Accept / Reject claim)

Body -json:

```
{
  "status": "ACCEPTED"
}
```

Response:

```
{ "claimId": "2", "status": "ACCEPTED" }
```

Authentication / Authorization

1. Registration - POST - `api/auth/register`

Description: Creates a new user with an email and password. The password is hashed using bcrypt before saving to the database.

2. Login - POST - `api/auth/login`

Description: Verifies the user's email and password. If the credentials are correct, return a JWT token for authentication.

3. JWT Tokens

Description: The JWT token is used to access protected routes.

Usage: Send the token in the request header:

- Authorization: Bearer <token>

4. Protected Routes

Description: Only authenticated users with a valid JWT token can access other endpoints

Database / Models and Relations:

- **User**
 - **Fields:** id (cuid), email (unique), password (hashed), name, phone?, avatarUrl?, timestamps
 - **Relations:** items[], claims[]
- **Item**
 - **Fields:** id (auto-increment), title, description?, type (LOST, FOUND), status (OPEN, CLAIMED, RESOLVED), location, date, category, userId, timestamps
 - **Relations:** user, photos[], claims[]
- **Photo**
 - **Fields:** id, url, publicId (Cloudnary), itemId
 - **Relations:** belongs to Item (cascade delete)
- **Claim**
 - **Fields:** id, itemId, userId, message?, status (PENDING, ACCEPTED, REJECTED), createdAt
 - **Relations:** belongs to Item | User

Enums:

- **ItemType:** LOST | FOUND
- **ItemStatus:** OPEN | CLAIMED | RESOLVED
- **ClaimStatus:** PENDING | ACCEPTED | REJECTED

Error Handling

Common responses:

- 400 – Bad Request (validation, wrong ID, invalid type, etc.)
- 401 – Unauthorized (no token or invalid/expired)
- 403 – Forbidden (not owner, can't claim own item)
- 404 – Not Found
- 409 – Conflict (user already exists, already claimed)
- 500 – Server Error (check console logs)

Most endpoints return { error: "message" } on failure.